RetroC: A Compiler for Retroactively Editable Dataflow Pipelines with Verified Record-Level Recomputation

Swaraj Joshi
Dept. of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India
24BDS0175

Johan Viji
Dept. of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India
24BCI0176

Sudeep Kumar Sahu
Dept. of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India
24BCT0160

***Abstract*—Data pipelines are routinely corrected after they have run: a faulty input record is fixed, a missing cleaning stage is inserted, or an obsolete stage is removed. Common practice either reruns the whole pipeline or reruns every stage downstream of the change, while incremental dataflow engines that avoid this fix the shape of the computation when it is built. This paper presents RetroC, a compiler and runtime for a pure, statically typed dataflow language in which three retroactive edits—modifying an input record, inserting a stage, and deleting a stage—are first-class, type-checked, atomic operations on a versioned dataflow graph called the Retroactive IR. After an edit, RetroC propagates a record-level work set: each stage visits only the records whose inputs changed, reuses per-record memo entries keyed by input and function hashes, updates an order-independent collection hash in constant time per changed record, and stops at the first stage whose output is unchanged. Every result can be checked against a full rerun of the textually rewritten program. On four synthetic workloads measured with a pinned, interleaved protocol (three warm-up runs, median of eleven), a single-record edit runs 9.6–10.1× faster than a full rerun from 10 k to 1 M records and 22.4× faster on a 12-stage pipeline, and 8.8–20.1× faster than stage-level invalidation. Correctness was confirmed on 10,000 randomized programs and edit scripts. We also report where the approach loses—batches of sequential edits, nested-loop joins, and edits that change every record—and what these results imply for a recompute-versus-patch cost model.**

Keywords—incremental computation, dataflow programming, compilers, intermediate representation, memoization, change propagation, differential testing

I. Introduction

Dataflow pipelines—chains of *map*, *filter*, *reduce* and *join* stages over collections of records—are the backbone of extract-transform-load jobs, feature engineering and scientific data analysis. They are rarely written once and left alone. A sensor log turns out to contain a corrupt reading; an analyst realises that values must be calibrated *before* they are aggregated; a debugging stage becomes obsolete. Each of these is a *retroactive* edit: it changes the history of a computation that has already run, and the user wants the outputs that the edited pipeline would have produced.

The usual way to obtain those outputs is to run the pipeline again. Batch engines recompute every stage for every record [1], and workflow orchestrators and build tools track dependencies between whole stages or files, so a single changed record invalidates every downstream stage in full [2]; [3]. Incremental computation research shows that far less work is needed when only a small part of the input changes [4]; [5], and incremental dataflow engines propagate *differences* through a dataflow graph with impressive efficiency [6]; [7]. These engines, however, fix the topology of the computation when it is built: inserting or deleting a stage is outside their model and requires rebuilding the computation.

This paper treats the pipeline's own structure as editable history. RetroC compiles a small, pure dataflow language into a versioned dataflow graph, the *Retroactive IR*, executes it once while recording per-record memo tables, and then accepts edit scripts that modify past input records, insert stages or delete stages. Each edit is type-checked and applied atomically, and its effect is propagated through the graph at the granularity of individual records.

A. Motivation

Two observations motivate the design. First, in a *pure* language a stage's output for a record is fully determined by that record's input value and the stage's function. Purity therefore makes record-level reuse sound without any runtime dependency tracing: if neither the input value nor the function changed, the cached output is still correct. Second, users who edit a pipeline need to *trust* and *understand* the result. An incremental engine that silently returns a different answer from a full rerun is worse than no engine at all, and one that cannot say what it recomputed is hard to debug. RetroC therefore pairs incremental propagation with a built-in verifier and a human-readable rebuild plan for every edit.

B. Problem Statement

Let P be a well-typed program, I its input data, and S the state left by executing P on I (outputs, memo tables and hashes). Given an edit e from the set {modify input record, insert stage, delete stage}, the problem is to produce outputs equal to those of a from-scratch run of the edited program, $Run(Apply(e, P), Apply(e, I))$, while doing work proportional to the records whose values actually change rather than to the size of the input times the number of stages. Invalid edits must be rejected with a diagnostic and must leave S unchanged.

C. Contributions

This paper makes the following contributions:

- A pure dataflow language and a versioned intermediate representation in which input modification, stage insertion and stage deletion are first-class, type-checked, atomic compiler operations with an exact undo on failure (Section IV).
- A record-level propagation algorithm that combines per-record memoization, an order-independent collection hash updated in constant time per changed record, and forwarding of only the records whose output changed, so that work per stage is proportional to the number of changed records (Sections IV–VI).

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

- An end-to-end verifier that compares propagated results with a fresh run of the textually rewritten program, and a differential testing harness that exercised 10,000 random programs and edit scripts (Section VII).
- An evaluation over 35 workload–edit scenarios against a full rerun and against stage-level invalidation, using a reproducible pinned and interleaved timing protocol, reporting both the scenarios where RetroC wins and those where it loses (Sections VIII–XI).

II. Literature Review

RetroC draws on several lines of work. This section summarises each and states how RetroC relates to it; Table I compares the approaches along the properties that matter for retroactive editing.

A. Batch Reprocessing and Workflow Systems

Data-parallel batch engines recompute results from their inputs, and some can recover lost partitions from recorded lineage [1]. Workflow management systems and build tools such as Make decide *which tasks* to rerun from a dependency graph between tasks or files [2]. Their unit of reuse is the whole task: a one-record change to a task's input reruns that task and every task after it in full. RetroC's stage-level baseline (B1 in Section VIII) models this behaviour.

B. Build Systems and Early Cutoff

Mokhov et al. characterise build systems by their scheduling and rebuilding strategies and identify *early cutoff*—stopping when a rebuilt target is unchanged—as a key property [3]. Build tools with dynamic dependencies and query-based compilers apply the same idea to tasks and to compiler queries. RetroC applies early cutoff at two levels: per record, through memo hits, and per stage, through the collection hash.

C. Self-Adjusting and Demand-Driven Computation

Self-adjusting computation records a dynamic dependence graph during execution and re-executes only the affected parts after an input change [4]; selective memoization keys memo entries on the values a computation actually uses [8]. Adapton makes incremental computation demand-driven and composable [5]. These systems are general-purpose and trace dependencies at runtime. RetroC is narrower: it exploits the static structure of a pure collection pipeline, so the dependency structure is the compiled graph itself and no tracing is needed. In exchange, it adds structural edits of the program, which are not part of these models.

D. Incremental Dataflow and View Maintenance

Differential dataflow represents collections as accumulated differences and propagates input changes through a fixed dataflow graph [6], built on the timely dataflow model of Naiad [7]. Database incremental view maintenance derives update rules for materialised views from query definitions [9]. Stream processors handle unbounded, time-advancing inputs [10]. All of these excel at input changes; none treats inserting or deleting an operator in an already-running computation as an incremental update.

E. Incremental Programs and Compilers

The incremental lambda calculus derives programs that map input changes to output changes for higher-order languages [11]. Reactive notebooks re-execute the cells that depend on an edited cell, which is again stage-level reuse. RetroC is a compiler in the conventional sense—lexer, parser, type checker, IR, bytecode—whose IR is designed to be *edited* after execution.

F. Retroactivity, Provenance and Testing

The term *retroactive* comes from retroactive data structures, which support inserting or deleting operations in the past of a data structure's history [12]; RetroC borrows the idea, not the technique. Its *explain* command relates to data provenance, which tracks why and where an output value came from [13]. Its verification strategy follows differential testing, which compares two implementations of the same specification on generated inputs [14]; [15], and metamorphic testing, which checks relations between related executions [16].

TABLE I. Comparison of Approaches to Recomputation After a Change

Approach	Input edits	Stage edits	Reuse unit	Verifier
Batch rerun [1]	rerun	rerun	none	–
Workflow / build [2]; [3]	yes	rerun stage	task	–
Self-adjusting [4]; [5]	yes	no	trace node	–
Differential dataflow [6]	yes	rebuild	difference	–
View maintenance [9]	yes	rebuild	delta	–
RetroC (this work)	**yes**	**yes**	**record**	**built in**

III. Research Gap

The literature offers either reuse at the level of whole stages or tasks, which is simple and correct but recomputes every record of every downstream stage, or fine-grained incremental computation over a computation whose shape is fixed when it is built. We found no system in which all of the following hold together:

- edits to past input records *and* structural edits that insert or delete stages are first-class, type-checked operations on the compiled program;
- reuse after such edits is tracked per record rather than per stage, with work proportional to the records that change;
- every edit produces an explicit rebuild plan stating what was recomputed and what was reused; and
- the incremental result is checked against a from-scratch run of the edited program.

RetroC is designed to fill this gap for a small but realistic pure dataflow language, and the evaluation in Section IX measures how far record-level reuse helps for each kind of edit.

IV. Proposed Methodology

A. The RetroC Language

A RetroC program is a sequence of typed `input` declarations, `let` stages, optional single-expression `fn` helpers and `output` declarations. Types are `int`, `float`, `bool`, `string`, arrays `[T]` and records `{f: T, ...}`. Collection builtins are `map`, `filter`, `reduce` (left fold), `tree_reduce` (associative fold) and `join` (equi-join); scalar helpers include

`length`, `to_float`, `min`, `max` and `clamp`. The language is pure: there is no mutation, no input or output inside lambdas, no randomness and no recursion, which guarantees termination and makes every stage a pure function of its inputs. Lambdas may appear only as arguments to builtins, and their parameter types are inferred from the builtin. Every user-facing error has a code (E101–E601), a line and a column.

Listing 1. A sensor pipeline and an edit script

```
input readings: [{id: int, temp_c: float,
                  valid: bool}] = [ ... ];
let ok    = filter(readings, r => r.valid);
let temp_f = map(ok, r => r.temp_c*1.8 + 32.0);
let sum_f = reduce(temp_f, 0.0, (a, t) => a + t);
let avg   = sum_f / to_float(length(temp_f));
output avg;

retro modify readings[1].temp_c = 22.0
retro insert before temp_f as cal :
      map(_, r => {id: r.id,
                   temp_c: r.temp_c + 0.5,
                   valid: r.valid})
retro delete cal
```

Listing 1 shows a pipeline and an edit script. The underscore in an inserted stage stands for the collection the new stage will read. Edits apply in order and each receives an edit identifier used by the history log and by provenance queries.

B. The Retroactive IR

Each top-level stage becomes one node of a directed acyclic graph. A node records a stable identifier, a kind (INPUT, MAP, FILTER, REDUCE, TREE_REDUCE, JOIN, SCALAR, OUTPUT), its input edges, its lambda or expression, a structural hash of that function (*fnHash*), a hash of its current output (*outHash*), a version counter and a memo table. A builtin nested inside another is split into a hidden stage so that every builtin application is one node. The graph keeps a topological order, recomputed with Kahn's algorithm after every structural edit [17]. Table II lists the invariants that the implementation maintains and that the tests check.

TABLE II. INVARIANTS OF THE RETROACTIVE IR

Id	Invariant
I1	The graph is acyclic and the stored order is topological.
I2	Node identifiers are unique and never reused.
I3	A memo entry is reused only if its input hash and function hash both match.
I4	A node's version increases if and only if its output hash changes.
I5	After propagation every node is clean and equals the pure function of its inputs.
I6	Every node type-checks against its inputs after every edit.
I7	Collections keep source order; RecordIds are stable through MAP and FILTER, so every collection is sorted by RecordId.
I8	Running with an empty memo gives the same outputs as running with a warm memo.

C. Edit Semantics

Table III summarises the three edits. Each edit first checks its preconditions, then rewrites the graph, recomputes the topological order and re-type-checks the affected stages. If any step fails, the rewrite is undone exactly—only the rewired edges and the added or removed node are restored—and the edit is reported with error E601. Because nothing else is mutated before these checks, the undo does not need to snapshot results or memo tables.

TABLE III. THE THREE RETROACTIVE EDITS

Edit	Precondition	Graph rewrite	Initial work set
Modify	input, record and field exist; literal has the field's type	replace the field; patch the input hash	the modified record
Insert	target is not an input; new lambda and target still type-check	new node N between P and target	all records of N
Delete	MAP/FILTER whose removal type-checks; not sole source of an output	rewire consumers to the stage's input	all records, memo-filtered

D. Record-Level Propagation

Propagation maintains a *work set* consisting of the dirty nodes, the nodes that must be rescanned in full, and, for each record-tracking node, the set of pending RecordIds. Nodes are visited in topological order. A MAP or FILTER stage with few pending records patches its output in place: for each pending record it locates the input and output positions by binary search on RecordId (invariant I7), looks up the memo with key (hash of the input value, *fnHash*), runs the lambda only on a miss, and inserts, replaces or removes the output record. When more than one eighth of the input is pending, a single sequential pass is cheaper and is used instead. Either way the stage reports exactly the records whose *output* changed, including records that appeared or disappeared, and only those are passed to its consumers. REDUCE and SCALAR nodes reuse their memo when the hash of all their inputs is unchanged; JOIN recomputes and renumbers its output, so its consumers are rescanned.

E. Order-Independent Collection Hash

Early cutoff needs a hash of each stage's output. Recomputing an order-dependent hash after every edit costs time proportional to the collection. RetroC instead uses a commutative sum of per-record terms. For a record with identifier r and value hash $h(v)$,

$$t(r, v) = mix(mix(r) \oplus h(v)), \tag{1}$$

where *mix* is the SplitMix64 finaliser [18] and $\oplus$ is bitwise exclusive or. The hash of a collection C combines its size with the sum of its terms modulo 2^{64}:

$$H(C) = F(|C|, \Sigma_{(r,v)\in C}\, t(r, v) \bmod 2^{64}), \tag{2}$$

where F is an FNV-style combination. Replacing the value of record r updates the stored sum S in constant time,

$$S' = S - t(r, v_{\text{old}}) + t(r, v_{\text{new}}), \tag{3}$$

and insertion or removal adds or subtracts one term. Order is still captured: by invariant I7 every collection is sorted by unique RecordIds, so the set of (r, v) pairs determines the sequence. Sum-based multiset hashes are a known construction [19]. They are not collision-resistant against an adversary, but RetroC's inputs are not adversarial. If the terms behave like independent uniform 64-bit values, the probability that any of q compared states collide is at most $q^2/2^{65}$ by the birthday bound, which is below 10^{-8} even for $q = 10^5$

comparisons. The verifier compares full values, not hashes, so a collision cannot go undetected in a verified run.

F. Correctness Argument

Claim. For every well-typed program P, input I and valid edit e, propagating e from the state of executing P on I yields the same outputs as running the edited program on the edited input from an empty memo.

Proof sketch. By induction over the topological order. The work set initially contains every record whose input value or stage function changed. When a stage is visited, a memo entry is reused only if both its input hash and its function hash match (I3); since stages are pure, equal input under an equal function yields an equal output. Records whose output is unchanged are not forwarded, and nodes not reached by the work set have unchanged inputs and functions, so their outputs are unchanged. The only gap is a hash collision, bounded above and closed by the verifier's full-value comparison.

G. Incremental Tree Reduction

A left fold must re-fold every element after a change. For associative combiners, `tree_reduce` stores a complete binary tree of partial results whose leaves are the records. Changing record k recomputes only the $\lceil \log_2 n \rceil$ internal nodes on the path from its leaf to the root: three combiner steps for $n = 8$ and ten for $n = 1{,}024$, both checked by unit tests.

V. System Architecture

RetroC is organised as a strict layered pipeline (Fig. 1). A module may include only modules to its left in the chain common → lexer → parser → sema → ir → bytecode → runtime → edit → reuse → verify → cli, so there are no dependency cycles. The front end turns source text into a typed program, the IR builder produces the Retroactive IR, lambda bodies are compiled to stack bytecode, and the graph executor performs the first run while filling memo tables. Edit scripts are parsed separately and applied by the edit engine; the propagator updates the graph and produces the rebuild plan; the verifier optionally re-runs the rewritten program. Table IV lists the modules with their size and responsibility.

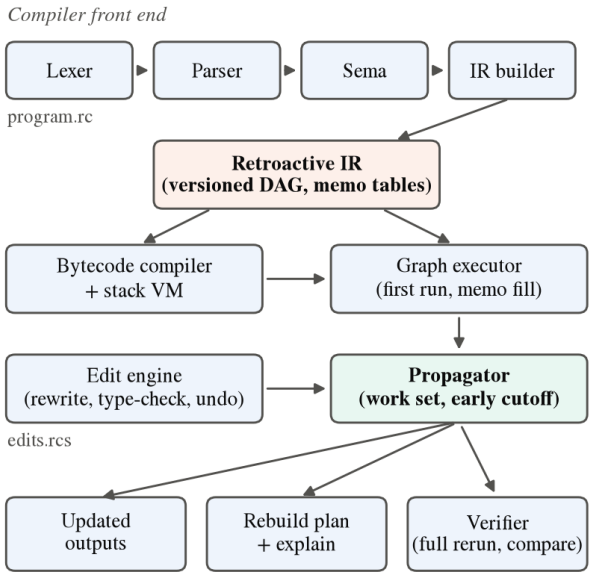


Fig. 1. Architecture of RetroC. The front end builds the Retroactive IR; the executor performs the first run; edits are applied as graph rewrites and propagated record by record; results feed the outputs, the rebuild plan and the verifier.

TABLE IV. Modules of the Implementation

Module	LOC	Responsibility
common	379	values, structural hashing, collection hash, errors
lexer	397	tokens; errors E101–E102
parser	1,208	recursive descent, lambda lookahead; E201
sema	901	types, scopes, purity; E301–E403
ir	1,611	graph, nodes, memo, topological sort, dump
bytecode	835	expression compiler and stack VM
runtime	904	first run, CSV loader, stage-reference analysis
edit	764	edit-script parser, source rewriting
reuse	1,261	propagation, rebuild plan, explain, B1 mode
verify	277	full-rerun comparison
cli	544	command-line driver

VI. Algorithm

Fig. 2 shows the complete control flow, from source text to verified outputs. Part A runs once. The program is lexed and parsed (step 1) and checked for types, scopes and purity (step 2); any error stops compilation with a code and a line:col position. The IR builder then produces the Retroactive IR in topological order (step 3), lambda bodies are compiled to bytecode (step 4), and the executor runs every stage once, filling the per-record memo tables and collection hashes (step 5).

Part B repeats for each edit in the script. The edit is read (step 6) and applied to a snapshot of the graph; inserts and deletes re-sort the stages with Kahn's algorithm [17], and the result is type-checked (step 7). An invalid edit, for example one that creates a cycle, restores the snapshot and reports E601, so the graph is unchanged. A valid edit seeds the work set W with the first affected stages and the pending record IDs (step 8). Stages are then visited in topological order (step 9). If more than an eighth of a stage's records are pending, one full pass with per-record memo checks is cheaper; otherwise only the pending records are patched (step 10). The collection hash is updated in $O(1)$ per changed record (step 11). If the stage's output hash is unchanged, propagation stops there (early cutoff); otherwise its version is increased and only the changed record IDs are forwarded to its consumers (step 12). When no dirty stages remain, the changed outputs are refreshed and the rebuild plan is printed (step 13). After the last edit, `retroc verify` can rewrite the source, rerun it from scratch and compare the outputs (step 14).

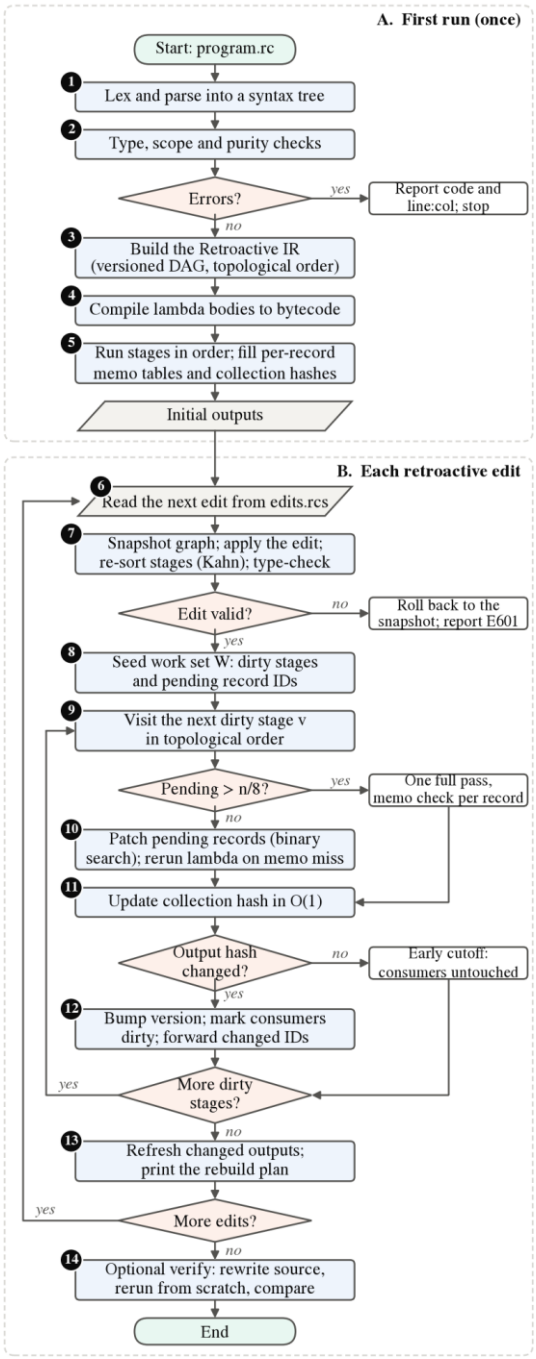


Fig. 2. End-to-end flowchart of RetroC. Part A compiles and runs the program once; Part B applies each retroactive edit and propagates it stage by stage. Steps 10 and 11 apply to MAP and FILTER stages; REDUCE, JOIN and scalar stages recompute unless their memo matches, and tree_reduce updates one leaf-to-root path.

Algorithm 1 gives the shared propagation loop used by all three edits (steps 8–12 of Fig. 2), and Algorithm 2 the patch step for a MAP or FILTER stage.

Algorithm 1. PROPAGATE(G, W)

```
Input: graph G, work set W = (dirty, fullScan,
       pending: node -> set of RecordIds)
for v in topological order of G:
  if v is INPUT or v not in W.dirty: continue
  h0 <- v.outHash
  if v is MAP or FILTER:
     changed <- PATCH(v, W)              (Alg. 2)
  else if v is TREE_REDUCE:
     update leaves in W.pending[v]; ascend
  else recompute v unless its memo matches
     the hashes of all its inputs
  if hash(v) != h0 or v is new:          (I4)
     v.version <- v.version + 1
     for c in consumers(v):
        W.dirty <- W.dirty + {c}
        if c tracks records: W.pending[c] += changed
        else W.fullScan <- W.fullScan + {c}
  (otherwise: early cutoff, consumers untouched)
refresh only the outputs whose hash changed
```

Algorithm 2. PATCH(v, W) for MAP / FILTER

```
K <- W.pending[v];  in <- input of v
if v in W.fullScan or |K| > |in|/8:
   one pass over in; memo check per record;
   compare with old output -> changed
else for r in K:
   x <- binarySearch(in, r)
   y <- binarySearch(out, r)
   if x exists and memo[r] matches
        (hash(x), fnHash): reuse          (I3)
   else if x exists: run lambda; store memo
   insert / replace / erase y; add or
   subtract t(r, v) in the hash sum    (3)
   if the output record changed: add r
return changed
```

Complexity. For k pending records in a stage of n records, the patch path performs k binary searches and at most k lambda evaluations, $O(k \log n)$ plus the cost of inserting or erasing vector elements when a filter admits or drops a record. The hash update is $O(1)$ per changed record. A linear REDUCE remains $O(n)$ whenever its input changes, while `tree_reduce` is $O(k \log n)$. Before this design, every visited stage cost $O(n)$ regardless of k, because it rehashed, re-looked-up and copied the whole collection.

VII. Implementation Details

A. Technology Stack

RetroC is written in C++17 (about 9,080 lines in `src/`, plus about 3,380 lines of tests and benchmark code) and built with CMake and Ninja using GCC 16.2.0. Warnings are treated as errors. The only third-party component is the single-header doctest framework; the CSV loader is written in-house. Golden tests are driven by small Python scripts.

B. Runtime Representation

Values are immutable tagged unions with structural equality, a 64-bit structural hash and deterministic floating-point printing, so identical inputs produce byte-identical output. Lambda bodies run on a stack virtual machine whose operand stack keeps 64 slots inline and spills to the heap only for very large literals, so ordinary expressions never allocate. Each stage result stores its collection, sorted by RecordId, together with the running sum behind its collection hash. A static analysis of the compiled bytecode records which stages are read by name inside expressions, so a collection is materialised for expressions only when one reads it, and only its size when it is read through `length`.

C. Command-Line Interface

The `retroc` driver exposes `dump-tokens`, `dump-ast`, `check`, `dump-ir`, `run`, `edit`, `verify` and `explain`. The `edit` command prints a rebuild plan per edit listing, for every visited stage, how many records were evaluated and reused;

`explain` lists the edits that changed a given output and the first stage each change entered.

D. Verification Infrastructure

Correctness is checked at four levels: doctest unit tests per module; golden tests that compare the exact output of every compiler stage, of edits and of `explain`; the `verify` command, which rewrites the source text with the edits, compiles and runs it from scratch and compares every output by full structural equality; and a differential tester [14]; [15]. The differential tester generates well-typed programs of six shapes (filter and map chains, a scalar reading a collection through `length`, branching graphs with `tree_reduce`, and joins) over 5 to 200 records, applies one to four random edits (60% modify, 20% insert, 20% delete), and also injects invalid edits that must be rejected without changing any state. A metamorphic test checks that inserting an identity map leaves outputs unchanged [16].

VIII. Experimental Setup

A. Dataset

The benchmarks use four synthetic, seeded workloads produced by a generator included with the artifact (Table V). W1 models a sensor log: temperatures are uniform in [−20, 120], humidity in [0, 100], and a reading is valid if its temperature lies in [−10, 100]; it is generated at 10 k, 100 k and 1 M records to study scaling. W2 joins 500 users with 2,000 orders whose amounts are uniform in [10, 500]. W3 is a deep pipeline of twelve alternating map and filter stages over 100 k records with values uniform in [1, 100]. W4 is reduce-heavy: one map and one fold over 1 M records with values uniform in [1, 1000]. Fixed seeds make every run reproducible.

TABLE V. Workloads

Id	Records	Pipeline
W1	10k / 100k / 1M	filter → map → map → filter → reduce
W2	500 + 2,000	join → map → filter → reduce
W3	100k	12 alternating map/filter → reduce
W4	1M	map → reduce

B. Edit Scenarios

Six edit scenarios are applied to each workload: E-A modifies one input record; E-B modifies 1% of the records, capped at 100 sequential edits; E-C inserts a map near the start of the pipeline; E-D inserts a map near the end; E-E deletes a middle stage; and E-F inserts an identity map. W4 has no E-E scenario because deleting its only middle stage would change the element type. This yields 35 scenarios.

C. Baselines

Two baselines run in the same runtime, so that only the recomputation strategy differs. **B0** reruns the edited program from scratch with an empty memo. **B1** is stage-level invalidation: any change to a stage's input recomputes every record of that stage without per-record memo, as a workflow or build system would, while keeping whole-stage early cutoff. **B2** is RetroC. A comparison with an external incremental engine is left to future work, because it would measure a different language and runtime.

D. Hardware and Protocol

All measurements were taken on an Intel Core i7-13620H laptop processor (6 performance and 4 efficiency cores, 16 threads) with 15.6 GB of RAM under Windows 11, using a release build with -O2. Each system receives three warm-up runs followed by eleven timed runs, and the median is reported; a system whose single run exceeds 10 s receives one warm-up and five timed runs [20]. Only propagation (B1, B2) or execution (B0) is timed; parsing, compilation, data loading and graph cloning are excluded. The process is pinned to one performance core at high priority, because the same work ran about twice as slowly on an efficiency core, and B0, B2 and B1 are timed round-robin so that slow drift such as thermal throttling affects all three alike. Every scenario is additionally run through the verifier.

E. Evaluation Metrics

- **Wall-clock time** per edit script, median of eleven runs (min and max are recorded).
- **Speedup over a full rerun**, B0/B2, and **over stage-level invalidation**, B1/B2.
- **Operations** recomputed and reused, where an operation is one lambda evaluation or one folded element, and **reuse** = reused / (recomputed + reused).
- **First-run overhead** of recording memo tables and history, and **memo size** relative to input size.
- **Correctness**: verifier result per scenario, unit and golden tests, and differential trials.

IX. Results

A. Single-Record Edits

Fig. 3 shows the speedup of RetroC over a full rerun when one input record is modified. On W1 it is 9.63×, 9.87× and 10.05× at 10 k, 100 k and 1 M records, and on the 12-stage W3 it is 22.39×. W4, which has only one map before its fold, gains 3.37×. On the small join workload W2 RetroC is slower than a rerun (0.40×). Fig. 4 shows the absolute times: all three systems scale linearly with input size, and RetroC stays an order of magnitude below both baselines at every size.

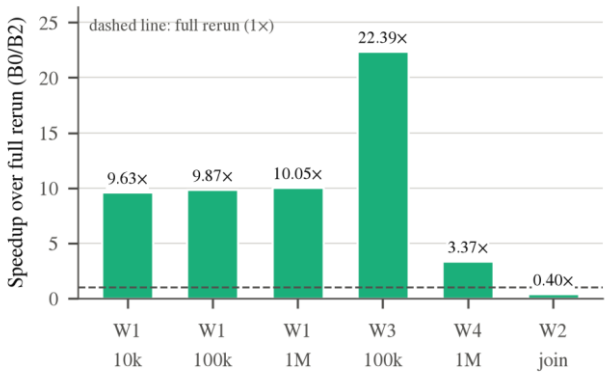


Fig. 3. Speedup of RetroC over a full rerun (B0/B2) for a single-record edit (E-A). Bars above the dashed line are faster than rerunning the pipeline.

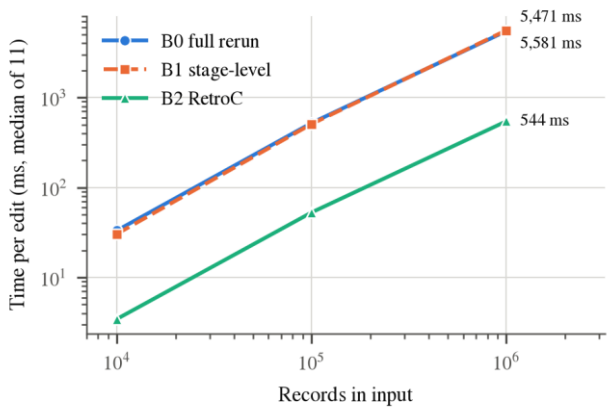


Fig. 4. Time per single-record edit on W1 versus input size (log–log). B0 and B1 nearly coincide; RetroC (B2) is about ten times lower at every size.

The ratio does not grow with input size because the final linear `reduce` must re-fold every record whose input changed. On W1-100k, 68,115 of the 68,119 operations recomputed by RetroC belong to that fold; only four are lambda evaluations in the map and filter stages. The fold is now the dominant cost, which is exactly what `tree_reduce` removes for associative combiners.

B. All Scenarios

Table VI lists all 35 scenarios and Fig. 5 compares the edit kinds on W1-100k and W3. RetroC beats a full rerun on single-record edits, on inserts near the end of the pipeline (3.0–4.8× on W1 and W3, 12.1× on W2), on identity inserts (2.5–5.0×, 12.0× on W2) and on deletes (1.4–2.3× on W1 and W3, 20.2× on W2). It breaks even or loses when an insert near the start changes every record (0.71–1.06×) and when 100 edits are applied one after another (0.03–0.23×), because each edit pays the fold again while a rerun pays it once.

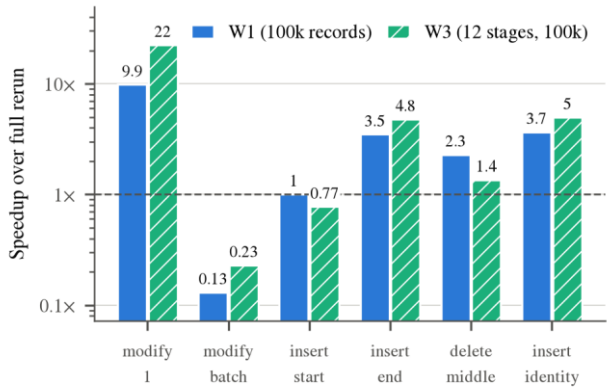


Fig. 5. Speedup over a full rerun (B0/B2) per edit scenario for W1 (100k records) and W3 (12 stages), logarithmic scale.

TABLE VI. MEDIAN TIME PER EDIT SCRIPT (MS) AND SPEEDUPS FOR ALL SCENARIOS

Work-load	Edit	B0	B1	B2	B0/B2	B1/B2
W1-10k	E-A	33.2	30.5	3.4	9.63	8.83
W1-10k	E-B	51.3	3975.2	413.5	0.12	9.61
W1-10k	E-C	59.6	49.8	56.5	1.06	0.88
W1-10k	E-D	55.5	14.8	17.2	3.23	0.86
W1-10k	E-E	39.1	16.0	18.0	2.17	0.89
W1-10k	E-F	56.1	23.6	14.3	3.92	1.64
W1-100k	E-A	520.9	507.6	52.8	9.87	9.61
W1-100k	E-B	537.0	39047.9*	4111.5	0.13	9.50
W1-100k	E-C	633.9	514.9	632.3	1.00	0.81
W1-100k	E-D	632.8	156.5	181.5	3.49	0.86
W1-100k	E-E	417.4	165.2	182.1	2.29	0.91
W1-100k	E-F	631.2	228.0	172.0	3.67	1.33
W1-1M	E-A	5471.2	5580.7	544.4	10.05	10.25
W1-1M	E-B	5409.4	–	37046.9*	0.15	–
W1-1M	E-C	8021.7	7636.6	9787.8	0.82	0.78
W1-1M	E-D	8546.2	2747.5	2810.7	3.04	0.98
W1-1M	E-E	4101.6	1672.4	2202.9	1.86	0.76
W1-1M	E-F	8333.4	3322.5	3398.4	2.45	0.98
W2	E-A	34.7	34.5	87.7	0.40	0.39
W2	E-B	35.0	683.3	1753.8	0.02	0.39
W2	E-C	36.3	31.6	103.6	0.35	0.31
W2	E-D	37.5	2.8	3.1	12.12	0.91
W2	E-E	36.6	1.9	1.8	20.23	1.03
W2	E-F	39.1	3.8	3.3	11.98	1.17
W3	E-A	1743.3	1566.7	77.9	22.39	20.12
W3	E-B	1757.5	150067.4*	7654.3	0.23	19.61
W3	E-C	2093.1	1847.0	2700.9	0.77	0.68
W3	E-D	2110.8	314.1	441.8	4.78	0.71
W3	E-E	1781.3	936.1	1312.9	1.36	0.71
W3	E-F	2101.5	343.6	422.0	4.98	0.81
W4	E-A	1617.6	1412.2	479.3	3.37	2.95
W4	E-B	1531.1	127994.5*	44762.4*	0.03	2.86
W4	E-C	3851.2	3448.5	5403.4	0.71	0.64
W4	E-D	2120.1	962.9	1140.5	1.86	0.84
W4	E-E	2074.1	467.4	631.2	3.29	0.74
W4	E-F	2074.1	467.4	631.2	3.29	0.74

* single run above 10 s: one warm-up and median of five runs. – not run (estimated over one hour). All 35 scenarios passed the verifier.

C. Record-Level Versus Stage-Level Reuse

Fig. 6 isolates the value of record granularity by comparing RetroC with stage-level invalidation. For edits to input records the difference is large: 9.5–9.6× on W1 and 19.6–20.1× on W3, for a single edit and for batches alike, because B1 recomputes every record of every downstream stage. For structural edits the picture is different. An inserted or re-wired stage must be evaluated on every record anyway, so record-level bookkeeping has little to save, and RetroC is 0.64–0.98× the speed of B1 except for identity inserts on the smaller inputs (1.33–1.64×).

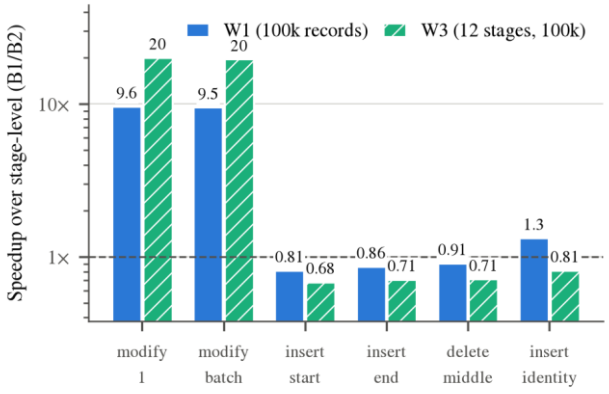


Fig. 6. Speedup of record-level reuse over stage-level invalidation (B1/B2) per edit scenario, logarithmic scale.

D. Effect of Record-Proportional Propagation

An earlier version of the propagator visited, hashed and copied every record of every dirty stage. Fig. 7 compares it with the record-proportional design on W1-100k using the same, earlier and less strict timing protocol (one warm-up and median of three, unpinned) for both. The single-record speedup rose from 1.28× to 5.42×, the late insert from 1.31× to 5.21×, and the delete from 0.86× to 2.20×, while operation counts were unchanged: the work avoided was bookkeeping, not computation. The same change removed three duplicated propagation loops and shrank the propagator from 2,067 to 1,261 lines.

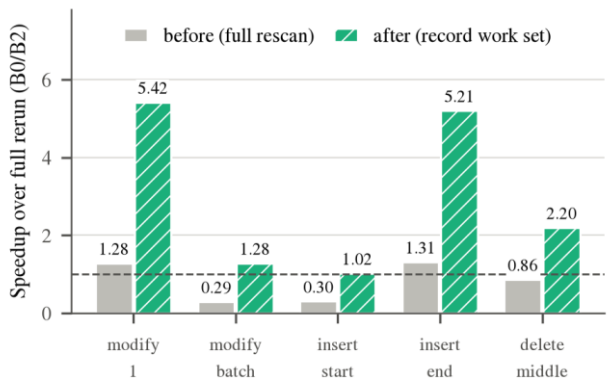


Fig. 7. Ablation on W1-100k: speedup over a full rerun before and after record-proportional propagation (same protocol; after = mean of two runs).

E. Overheads

Recording memo tables and history during the first run of W1-100k raises its time from 508.3 ms to 725.5 ms, an overhead of 42.7%. The memo tables occupy 36.18 MB for 1.73 MB of CSV input, about 21× the input size, because each record's memo entry stores two hashes and a copy of the output value. A single-record edit recovers the first-run overhead many times over: on W1-100k it saves about 468 ms per edit compared with a rerun.

F. Correctness Results

TABLE VII. CORRECTNESS EVIDENCE

Check	Result
Unit tests (doctest)	52 cases, 687 assertions pass
Golden tests (6 suites)	all pass, outputs unchanged
Differential trials, 5 seeds	10,000 / 10,000 match rerun
Invalid-edit injection	all rejected, state unchanged
Planted bug (mutation)	detected at trial 16
Verifier on benchmarks	35 / 35 scenarios pass

Table VII summarises the correctness evidence. To check that the differential tester can detect real propagation errors, we planted a defect—a record removed from a stage's output was not forwarded to its consumers—and the tester reported a mismatch at trial 16.

X. COMPARISON WITH EXISTING AND BASELINE METHODS

Against the two baselines, the results divide cleanly by the kind of edit. For edits to input records, RetroC improves on a full rerun by up to 22× and on stage-level invalidation by up to 20×; this is the case incremental systems are built for, and record granularity is what delivers the gain. For structural edits RetroC improves on a full rerun by 1.4–20× whenever the edit leaves part of the pipeline untouched, but performs about as well as stage-level invalidation, because the edited stage and its consumers must see every record. RetroC loses to both baselines on the join workload and on sequential batches of edits.

Qualitatively (Table I), RetroC is the only approach among those surveyed in which inserting or deleting a stage is an incremental, type-checked operation on an already-executed computation, and the only one with a built-in check against a full rerun. Differential dataflow and incremental view maintenance are mature, highly optimised engines for input changes [6]; [9]; we did not measure them, and on input edits at scale we expect them to be faster than RetroC's single-threaded interpreter. RetroC's contribution is the edit model and its verified, explainable record-level propagation, not raw throughput.

XI. DISCUSSION

Where the time goes. After the redesign, the remaining cost of a single-record edit on W1 and W4 is the linear fold. This is the expected Amdahl-style limit: the more stages a pipeline has before its fold, the larger the share of work that record-level reuse removes, which is why the 12-stage W3 gains 22× while the two-stage W4 gains 3.4×.

Batches. Applying 100 edits one at a time re-folds the input 100 times. A rerun pays for one fold, so RetroC loses by an order of magnitude. Either an associative fold (`tree_reduce`), or a cost model that coalesces a batch and switches to a rerun when the batch is large, would remove this loss.

Joins. The join operator is a nested loop with a per-pair memo. On W2 the memo lookup for each of one million pairs costs more than simply comparing the keys, so memoization makes the join slower. A hash join with per-key incremental maintenance is the natural fix.

Structural edits. Record-level reuse after an insert or delete helps only for the records whose values do not change. When an inserted stage changes every record, nothing can be reused, and the per-record bookkeeping becomes overhead. These scenarios should be routed to stage-level recomputation by the cost model.

Measurement. Hybrid laptop processors produced up to threefold timing differences between otherwise identical runs, depending on whether the operating system scheduled the benchmark on a performance or an efficiency core and on thermal state. Pinning to one performance core and timing all systems round-robin reduced the spread within a scenario to about ±10%. We recommend both practices for incremental-computation benchmarks on consumer hardware.

Threats to validity. The workloads are synthetic, the language is small, all measurements come from one machine, and W2 is much smaller than a realistic join because the join is a nested loop. The results support claims about this design on these workloads, not general performance claims.

XII. LIMITATIONS

- A linear `reduce` re-folds all records after any change, which caps single-edit speedups; only associative folds can use `tree_reduce`.
- Sequential batches of edits, nested-loop joins and inserts that change every record are slower than, or only equal to, a full rerun.
- On structural edits RetroC is roughly even with stage-level invalidation, not better.
- A lambda that reads another stage by name (for example `x + total`) is not recorded as a dependency of its stage, so changing that stage can leave stale cached results; fixing this requires extending the IR so that such references become inputs and enter the memo key.
- Memo tables use about 21× the input size, and the first run is 42.7% slower; execution is single-threaded and in memory.

- Edits cannot add or remove input records, replace a stage in one step, or change control flow.
- Memory-safety sanitizers could not be run with the MinGW toolchain used.

XIII. Conclusion

RetroC shows that modifying past inputs, inserting stages and deleting stages can be handled as typed, atomic, first-class operations of a compiler for a pure dataflow language, and that their effects can be propagated record by record with work proportional to what changes. A single-record edit runs about ten times faster than a full rerun on a sensor pipeline from 10 k to 1 M records, 22 times faster on a 12-stage pipeline, and up to 20 times faster than stage-level invalidation, and every result matched a full rerun across unit, golden and benchmark tests and 10,000 random edit scripts. The measurements are equally clear about the remaining losses—sequential batches, joins and edits that touch every record—which point directly to the next steps.

XIV. Future Work

- **Cost model:** choose between patching and recomputing per stage and per batch, using the number of pending records and static cost estimates, to remove every loss reported in Section IX.
- **Incremental folds:** maintain invertible or associative reductions incrementally so that speedups grow with input size.
- **Hash join:** replace the nested-loop join with a hash join maintained per key.
- **Stage-reference dependencies:** record stages read inside lambdas as inputs and include their hashes in memo keys.
- **External baseline:** compare input-edit performance with an established incremental dataflow engine.
- **Richer edits and parallelism:** record insertion and deletion, stage replacement, and parallel propagation across independent branches.

Acknowledgment

[To be completed by the authors: supervisors, institution, funding.]

References

[1] M. Zaharia *et al.*, “Apache Spark: A unified engine for big data processing,” *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016, doi: 10.1145/2934664.

[2] J. Liu, E. Pacitti, P. Valduriez, and M. Mattoso, “A survey of data-intensive scientific workflow management,” *J. Grid Comput.*, vol. 13, no. 4, pp. 457–493, 2015, doi: 10.1007/s10723-015-9329-8.

[3] A. Mokhov, N. Mitchell, and S. Peyton Jones, “Build systems à la carte,” *Proc. ACM Program. Lang.*, vol. 2, no. ICFP, pp. 1–29, 2018, doi: 10.1145/3236774.

[4] U. A. Acar, G. E. Blelloch, and R. Harper, “Adaptive functional programming,” in *Proc. 29th ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang. (POPL)*, 2002, pp. 247–259, doi: 10.1145/503272.503296.

[5] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster, “Adapton: Composable, demand-driven incremental computation,” in *Proc. 35th ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2014, pp. 156–166, doi: 10.1145/2594291.2594324.

[6] M. Abadi, F. McSherry, and G. D. Plotkin, “Foundations of differential dataflow,” in *Foundations of Software Science and Computation Structures (FoSSaCS)*, Lecture Notes in Computer Science, vol. 9034. Berlin, Germany: Springer, 2015, pp. 71–83, doi: 10.1007/978-3-662-46678-0_5.

[7] D. G. Murray, F. McSherry, R. Isaacs, M. Isard, P. Barham, and M. Abadi, “Naiad: A timely dataflow system,” in *Proc. 24th ACM Symp. Oper. Syst. Princ. (SOSP)*, 2013, pp. 439–455, doi: 10.1145/2517349.2522738.

[8] U. A. Acar, G. E. Blelloch, and R. Harper, “Selective memoization,” in *Proc. 30th ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang. (POPL)*, 2003, pp. 14–25, doi: 10.1145/604131.604133.

[9] J. A. Blakeley, P.-Å. Larson, and F. W. Tompa, “Efficiently updating materialized views,” in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1986, pp. 61–71, doi: 10.1145/16894.16861.

[10] A. Toshniwal *et al.*, “Storm@twitter,” in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2014, pp. 147–156, doi: 10.1145/2588555.2595641.

[11] Y. Cai, P. G. Giarrusso, T. Rendel, and K. Ostermann, “A theory of changes for higher-order languages: Incrementalizing λ-calculi by static differentiation,” in *Proc. 35th ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2014, pp. 145–155, doi: 10.1145/2594291.2594304.

[12] E. D. Demaine, J. Iacono, and S. Langerman, “Retroactive data structures,” *ACM Trans. Algorithms*, vol. 3, no. 2, Art. no. 13, 2007, doi: 10.1145/1240233.1240236.

[13] P. Buneman, S. Khanna, and W.-C. Tan, “Why and where: A characterization of data provenance,” in *Database Theory—ICDT 2001*, Lecture Notes in Computer Science, vol. 1973. Berlin, Germany: Springer, 2001, pp. 316–330, doi: 10.1007/3-540-44503-X_20.

[14] J. Chen *et al.*, “A survey of compiler testing,” *ACM Comput. Surv.*, vol. 53, no. 1, pp. 1–36, 2020, doi: 10.1145/3363562.

[15] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *Proc. 32nd ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2011, pp. 283–294, doi: 10.1145/1993498.1993532.

[16] T. Y. Chen *et al.*, “Metamorphic testing: A review of challenges and opportunities,” *ACM Comput. Surv.*, vol. 51, no. 1, pp. 1–27, 2018, doi: 10.1145/3143561.

[17] A. B. Kahn, “Topological sorting of large networks,” *Commun. ACM*, vol. 5, no. 11, pp. 558–562, 1962, doi: 10.1145/368996.369025.

[18] G. L. Steele, D. Lea, and C. H. Flood, “Fast splittable pseudorandom number generators,” in *Proc. ACM Int. Conf. Object-Oriented Program. Syst. Lang. Appl. (OOPSLA)*, 2014, pp. 453–472, doi: 10.1145/2660193.2660195.

[19] D. Clarke, S. Devadas, M. van Dijk, B. Gassend, and G. E. Suh, “Incremental multiset hash functions and their application to memory integrity checking,” in *Advances in Cryptology—ASIACRYPT 2003*, Lecture Notes in Computer Science, vol. 2894. Berlin, Germany: Springer, 2003, pp. 188–207, doi: 10.1007/978-3-540-40061-5_12.

[20] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Program. Syst. Lang. Appl. (OOPSLA)*, 2007, pp. 57–76, doi: 10.1145/1297027.1297033.